

Taskfile Automation & BigQuery Query Pipeline

Otomasi distribusi & pelaporan harian task file collection agent, berbasis Python (Jupyter Notebook) dan Google BigQuery

Daily task-file distribution & reporting automation for collection agents, built with Python (Jupyter Notebook) and Google BigQuery.

Perusahaan	Astra Welab Digital Arta (Maucash)
Peran	Data / Automation Engineer
Tools	Python, Jupyter Notebook, Google BigQuery, APScheduler, Gmail SMTP
Status	Perusahaan sudah tidak beroperasi — dokumentasi disusun ulang untuk keperluan portofolio

■ Catatan Keamanan

Kredensial asli (App Password, alamat pengirim, dan alamat email penerima internal) telah dihapus/disamarkan dari dokumen ini dan diganti dengan placeholder generik. Struktur query dan nama tabel BigQuery ditampilkan sebagaimana pada source asli untuk tujuan dokumentasi teknis.

■ Security Note

Original credentials (App Password, sender address, and internal recipient addresses) have been removed/masked from this document and replaced with generic placeholders. Query structure and BigQuery table names are shown as-is from the original source for technical documentation purposes.

■ Dokumen Terkait

Tabel `batch_distribution_ownership_[YYMM]` yang dibaca sebagai `TABLE_DIST` pada notebook ini dihasilkan oleh pipeline scoring & mapping bulanan yang terpisah. Detail teknisnya dibahas di dokumen "Query: Mapping Assignment Taskfile for Agent Collection".

The `batch_distribution_ownership_[YYMM]` table read as `TABLE_DIST` in this notebook is produced by a separate monthly scoring & mapping pipeline. Technical details are covered in the "Query: Mapping Assignment Taskfile for Agent Collection" document.

1. Ringkasan Pekerjaan

Proyek ini adalah pipeline automation harian yang menggantikan proses distribusi task file collection agent yang sebelumnya dilakukan manual. Sistem berjalan di Jupyter Notebook dan melakukan query ke Google BigQuery, menghitung alokasi data per agent berdasarkan kuota, menggabungkan hasil, lalu mengirimkannya otomatis via email dalam format CSV setiap hari pada jam terjadwal.

This project is a daily automation pipeline that replaced the previously manual distribution of collection-agent task files. The system runs in Jupyter Notebook, queries Google BigQuery, calculates each agent's data allocation based on quota, merges the results, and automatically emails them out as a CSV attachment every day at a scheduled time.

- Menghilangkan proses manual query & distribusi task file harian ke tiap agent collection.
Eliminates the manual process of querying and distributing daily task files to each collection agent.
- Memastikan pembagian data adil & konsisten antar agent menggunakan sistem kuota & windowing berbasis row number (rn_assign).
Ensures fair and consistent data distribution across agents using a quota and row-number-based windowing system (rn_assign).
- Output terkirim otomatis setiap hari tanpa intervensi manual, lengkap dengan file CSV terlampir.
Output is delivered automatically every day with no manual intervention, complete with an attached CSV file.

2. Alur Proses (Pipeline Flow)

#	Tahap	Deskripsi
1	Autentikasi BigQuery	Membuat BigQuery client menggunakan service account credentials, menentukan project, lokasi dataset (asia-southeast2 / Jakarta), dan folder output CSV. <i>Creates a BigQuery client using service account credentials, sets the project, dataset location (asia-southeast2 / Jakarta), and CSV output folder.</i>
2	Definisi Kuota Agent	Menetapkan daftar agent dan kuota harian masing-masing — agent biasa mendapat 400 baris data/hari, agent Team Leader mendapat 300 baris/hari. <i>Defines the agent list and each agent's daily quota — regular agents get 400 rows/day, Team Leader agents get 300 rows/day.</i>
3	Perhitungan Window (rn_assign)	Menghitung rentang row number (start–end) yang menjadi jatah tiap agent pada hari berjalan, dihitung berdasarkan selisih hari dari tanggal mulai fase (START_DATE) dikali kuota harian. <i>Calculates the row-number range (start–end) allotted to each agent for the current day, based on days elapsed since the phase start date (START_DATE) multiplied by the daily quota.</i>
4	Query & Fetch Data	Menjalankan parameterized query ke BigQuery — JOIN antara tabel task harian dan tabel distribusi kepemilikan, difilter berdasarkan agent_id dan rentang rn_assign. <i>Runs a parameterized query against BigQuery — a JOIN between the daily task table and the ownership-distribution table, filtered by agent_id and rn_assign range.</i>
5	Preview per Agent	Menampilkan hasil query per agent (jumlah baris, rentang rn_assign) sebagai langkah verifikasi sebelum data digabung/dikirim. <i>Displays per-agent query results (row count, rn_assign range) as a verification step before data is merged and sent.</i>

6	Ringkasan & Deteksi Duplikat	Merangkum jumlah baris, keunikan rn_assign, dan keunikan loan_number per agent untuk quality check sebelum data final dikirim. <i>Summarizes row counts and rn_assign / loan_number uniqueness per agent as a quality check before the final data is sent.</i>
7	Penggabungan Data	Menggabungkan hasil ketiga agent menjadi satu dataset, dengan fallback membaca CSV harian dari folder output jika data preview tidak tersedia. Kolom internal (agent_id, rn_assign) disembunyikan dari output final. <i>Merges the three agents' results into one dataset, with a fallback to reading the daily CSV from the output folder if preview data is unavailable. Internal columns (agent_id, rn_assign) are hidden from the final output.</i>
8	Pengiriman Email Otomatis	Menyimpan hasil gabungan sebagai CSV (delimiter ' '), lalu mengirim via SMTP Gmail dengan CSV sebagai lampiran ke daftar penerima terkait. <i>Saves the merged result as a CSV (pipe-delimited), then sends it via Gmail SMTP with the CSV as an attachment to the relevant recipient list.</i>
9	Penjadwalan Harian	Menjalankan seluruh proses secara otomatis setiap hari pada jam tetap menggunakan background scheduler (cron-style), tanpa perlu dijalankan manual. <i>Runs the entire process automatically every day at a fixed time using a background scheduler (cron-style), with no manual trigger needed.</i>

3. Query & Integrasi BigQuery

Autentikasi dilakukan menggunakan service account JSON, dengan project ID diambil otomatis dari kredensial. Client BigQuery digunakan untuk seluruh query di sepanjang notebook.

Authentication is done using a service-account JSON key, with the project ID pulled automatically from the credentials. The BigQuery client is reused for every query throughout the notebook.

```
from google.cloud import bigquery
from google.oauth2 import service_account

SERVICE_ACCOUNT_FILE = os.path.join(os.getcwd(), "[SERVICE_ACCOUNT_KEY].json")
credentials = service_account.Credentials.from_service_account_file(SERVICE_ACCOUNT_FILE)
PROJECT_ID = credentials.project_id

BQ_LOCATION = "asia-southeast2" # Jakarta
DATASET      = "collection_efforts"
TABLE_TASK   = "daily_taskfile"
TABLE_DIST   = "batch_distribution_ownership_2509"

client = bigquery.Client(project=PROJECT_ID, credentials=credentials)
```

3.1 Logika Pembagian Kuota per Agent

Setiap agent memiliki kuota harian berbeda tergantung levelnya — agent Team Leader mendapat kuota lebih kecil (300) karena memiliki tanggung jawab tambahan, sementara agent reguler mendapat kuota standar (400).

Each agent has a different daily quota depending on their level — Team Leader agents get a smaller quota (300) due to their additional responsibilities, while regular agents get the standard quota (400).

```
AGENTS = ["[AGENT_ID_1]", "[AGENT_ID_2]", "[AGENT_ID_3]"]
LEADERS = {"[AGENT_ID_3]"} # agent milik TL, kuota lebih kecil

DEFAULT_QUOTA = 400
LEADER_QUOTA = 300

def quota_for(agent_id: str) -> int:
    return LEADER_QUOTA if agent_id in LEADERS else DEFAULT_QUOTA
```

3.2 Windowing Berbasis Row Number

Untuk memastikan setiap agent mendapat data yang berbeda dan tidak tumpang tindih setiap harinya, sistem menghitung window `rn_assign` (rentang row number) berdasarkan selisih hari sejak tanggal mulai fase distribusi. Semakin jauh dari `START_DATE`, window bergeser maju sebesar nilai kuota harian.

To ensure each agent gets different, non-overlapping data every day, the system calculates an `rn_assign` window (row-number range) based on days elapsed since the distribution phase's start date. The further from `START_DATE`, the further the window shifts forward, by the daily quota amount.

```
START_DATE = date(2025, 9, 10)

def rn_window(run_d, q):
    day_idx = max(0, (run_d - START_DATE).days)
    start_rn = day_idx * q + 1
    end_rn   = start_rn + q - 1
    return start_rn, end_rn
```

Contoh: pada kuota 400/hari, hari ke-0 mendapat rn_assign 1–400, hari ke-1 mendapat 401–800, dan seterusnya. Logika ini divalidasi dengan unit assertion sebelum digunakan di pipeline utama.

Example: at a quota of 400/day, day 0 gets rn_assign 1–400, day 1 gets 401–800, and so on. This logic is validated with unit assertions before being used in the main pipeline.

3.3 Query Utama (Parameterized JOIN)

Data diambil dengan JOIN antara tabel task harian dan tabel distribusi kepemilikan, menggunakan parameterized query (bukan string formatting langsung) untuk mencegah SQL injection dan menjaga query plan tetap efisien.

Data is retrieved via a JOIN between the daily task table and the ownership-distribution table, using a parameterized query (rather than direct string formatting) to prevent SQL injection and keep the query plan efficient.

```
def fetch_agent_chunk(agent_id: str, start_rn: int, end_rn: int):
    sql = f"""
    SELECT t.*, d.rn_assign
    FROM `{PROJECT_ID}`.{DATASET}.{TABLE_TASK}` AS t
    JOIN `{PROJECT_ID}`.{DATASET}.{TABLE_DIST}` AS d
      ON t.loan_number = d.loan_number
    WHERE d.agent_id = @agent
      AND d.rn_assign BETWEEN @s AND @e
    ORDER BY d.rn_assign
    """
    params = [
        bigquery.ScalarQueryParameter("agent", "STRING", agent_id),
        bigquery.ScalarQueryParameter("s", "INT64", start_rn),
        bigquery.ScalarQueryParameter("e", "INT64", end_rn),
    ]
    job = client.query(sql, job_config=bigquery.QueryJobConfig(query_parameters=params),
                       location=BQ_LOCATION)
    return job.result().to_dataframe()
```

4. Preview, Ringkasan & Validasi Data

Sebelum data digabung dan dikirim, pipeline menjalankan tahap preview per agent serta ringkasan kualitas data (jumlah baris, rentang `rn_assign`, deteksi duplikat pada `rn_assign` maupun `loan_number`). Tahap ini berfungsi sebagai quality gate manual sebelum email dikirim.

Before data is merged and sent, the pipeline runs a per-agent preview step plus a data-quality summary (row count, `rn_assign` range, duplicate detection on both `rn_assign` and `loan_number`). This acts as a manual quality gate before the email goes out.

```
def run_all_agents_preview():
    previews = {}
    for agent in AGENTS:
        q = quota_for(agent)
        s, e = rn_window(run_d, q)
        df = fetch_agent_chunk(agent, s, e)
        previews[agent] = df
        # log jumlah baris, rn_min, rn_max per agent
    return previews
```

Fungsi `summarize_preview()` menghasilkan tabel ringkasan berisi jumlah baris, rentang `rn_assign`, jumlah nilai unik, dan jumlah baris duplikat — baik dari sisi `rn_assign` maupun `loan_number` — sebagai sanity check sebelum data digabungkan lintas agent.

The `summarize_preview()` function produces a summary table with row counts, `rn_assign` range, unique-value counts, and duplicate-row counts — for both `rn_assign` and `loan_number` — as a sanity check before merging data across agents.

5. Automation — Pengiriman Email & Penjadwalan

Tahap akhir pipeline menggabungkan data ketiga agent (dengan fallback membaca file CSV harian bila data preview tidak tersedia di memori), menyembunyikan kolom internal, lalu mengirim hasil sebagai lampiran CSV melalui SMTP Gmail secara terjadwal.

The pipeline's final stage merges the three agents' data (falling back to reading the daily CSV file if preview data isn't available in memory), hides internal columns, then sends the result as a CSV attachment via Gmail SMTP on a schedule.

5.1 Konfigurasi Pengiriman (disamarkan)

5.1 Delivery Configuration (masked)

```
sender_email    = "[SENDER_EMAIL]"
sender_password = "[APP_PASSWORD]" # Gmail App Password, 16 karakter
recipients = [
    "[internal - Tim Collection]",
    "[internal - Team Lead]",
    "[eksternal - stakeholder terkait]",
    # ... daftar penerima disamarkan untuk dokumentasi ini
]
```

5.2 Fungsi Pengiriman Email

5.2 Email Delivery Function

```
def send_taskfile_email():
    # susun subject & nama file berbasis tanggal (WIB)
    # simpan CSV dengan delimiter '|'
```

```
# attach CSV ke email (MIMEApplication)
with smtplib.SMTP("smtp.gmail.com", 587, timeout=30) as server:
    server.starttls()
    server.login(sender_email, sender_password)
    server.sendmail(sender_email, recipients, msg.as_string())
```

5.3 Penjadwalan Otomatis (APScheduler)

5.3 Automatic Scheduling (APScheduler)

Seluruh proses di atas dijalankan otomatis setiap hari tanpa perlu trigger manual, menggunakan background scheduler dengan trigger cron.

The entire process above runs automatically every day with no manual trigger, using a background scheduler with a cron trigger.

```
from apscheduler.schedulers.background import BackgroundScheduler

scheduler = BackgroundScheduler(
    timezone=TZ,
    job_defaults=dict(coalesce=True, max_instances=1, misfire_grace_time=3600)
)
scheduler.add_job(send_taskfile_email, trigger="cron", hour=15, minute=45,
                  id="email_taskfile_daily")
scheduler.start()
```

Konfigurasi cron di atas berjalan pada jam 15:45 UTC (≈ 22:45 WIB); komentar pada source asli menyebutkan target 07:45 WIB — kemungkinan ada penyesuaian timezone yang perlu diverifikasi ulang bila notebook ini dijalankan kembali.

Note: the cron config above runs at 15:45 UTC (≈ 22:45 WIB); a comment in the original source mentions a 07:45 WIB target — there may be a timezone mismatch worth re-verifying if this notebook is run again.

6. Ringkasan Tech Stack

Kategori	Tools / Library
Data Warehouse	Google BigQuery (parameterized query, service account auth)
Bahasa & Environment	Python 3, Jupyter Notebook
Data Processing	pandas (DataFrame manipulation, dedup check, CSV export)
Automation & Scheduling	APScheduler (BackgroundScheduler, cron trigger)
Notifikasi / Delivery	smtplib + email.mime (Gmail SMTP, attachment CSV)
Timezone Handling	pytz (Asia/Jakarta)

7. Kontribusi & Dampak

- Merancang dan membangun end-to-end pipeline dari query BigQuery hingga pengiriman laporan otomatis.
Designed and built the end-to-end pipeline, from BigQuery querying to automated report delivery.
- Mendesain logika pembagian kuota & windowing data yang adil antar agent, mencegah duplikasi penugasan.
Designed the quota and data-windowing logic to ensure fair distribution across agents and prevent duplicate assignments.
- Menambahkan lapisan validasi (preview, summary, duplicate check) sebagai quality gate sebelum data dikirim.
Added a validation layer (preview, summary, duplicate check) as a quality gate before data is sent.
- Mengotomasi proses yang sebelumnya manual menjadi terjadwal harian tanpa intervensi tim.
Automated a previously manual process into a daily scheduled job requiring no team intervention.

Dokumen ini disusun ulang dari source code internal (Jupyter Notebook) untuk keperluan portofolio pribadi. Kredensial, nama sistem sensitif, dan data kontak telah disamarkan.

This document was reconstructed from internal source code (Jupyter Notebook) for personal portfolio purposes. Credentials, sensitive system names, and contact data have been masked.